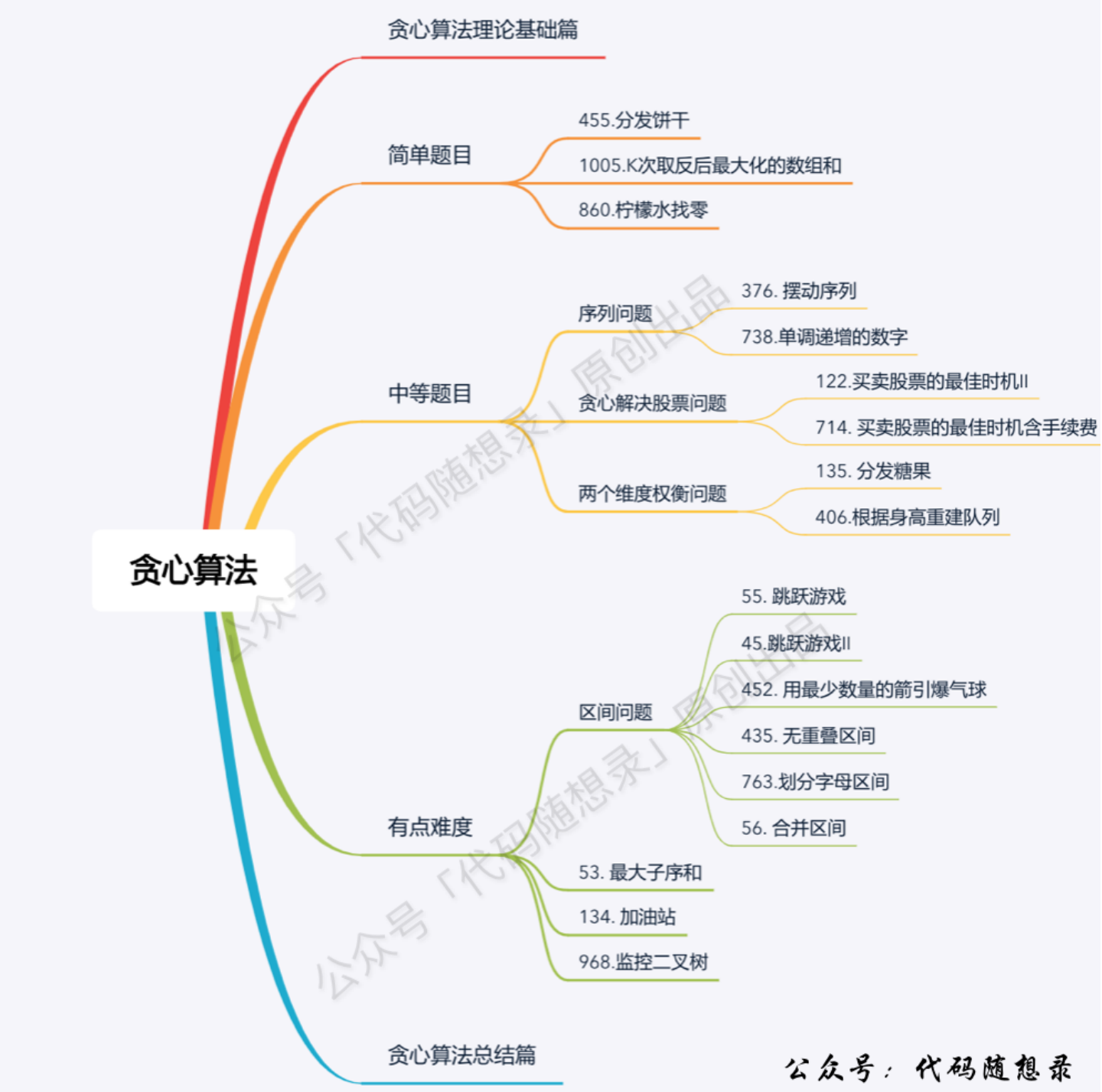
# 贪心算法

## 三.贪心算法




### 455.分发饼干

> 让胃口小的孩子先吃能喂饱的饼干尺寸！！！

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子 `i` ，都有一个胃口值 `g[i]` (，)这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 `j` ，都有一个尺寸 `s[j]` ()。如果 `s[j] >= g[i]` ，我们可以将这个饼干 `j` 分配给孩子 `i` ，这个孩子会得到满足。你的目标是满足尽可能多的孩子，并输出这个最大数值。

**示例 1:**

**输入:** g = [1,2,3], s = [1,1]
**输出:** 1
**解释:**
你有三个孩子和两块小饼干，3 个孩子的胃口值分别是：1,2,3。
虽然你有两块小饼干，由于他们的尺寸都是 1，你只能让胃口值是 1 的孩子满足。
所以你应该输出 1。

代码块

```
class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        #g[i]:第i个孩子的胃口
        #s[j]:第j块饼干的尺寸
        g.sort()
        s.sort()
        i,j =0,0
        n,m = len(g),len(s)
        while i<n and j<m:
            if s[j]>=g[i]:
                i+=1     #表示可以喂饱g[i]孩子，开始喂下一个
            j+=1
        return i
```

## 53.最大子数组和

给你一个整数数组 `nums` ，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组是数组中的一个连续部分。

**示例 1：**

**输入：** nums = [-2,1,-3,4,-1,2,1,-5,4]
**输出：** 6
**解释：** 连续子数组 [4,-1,2,1] 的和最大，为 6 。

代码块

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
```

```
        n = len(nums)
        dp =[0]*n  #dp[i]:表示以nums[i]结尾的子数组的最大和
        dp[0] = nums[0]
        for i in range(1,n):
            if dp[i-1]<=0:
                dp[i] = nums[i]
            else:
                dp[i] = dp[i-1]+nums[i]
        return max(dp)
```

## 55.跳跃游戏

给你一个**非负**整数数组 `nums` ，你最初位于数组的 **第一个下标** 。数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标，如果可以，返回 `true` ；否则，返回 `false` 。

**示例 1：**

**输入：** nums = [2,3,1,1,4]
**输出：** true
**解释：** 可以先跳 1 步，从下标 0 到达下标 1, 然后再从下标 1 跳 3 步到达最后一个下标。

**解题思路：**

1.如果某一个作为 起跳点 的格子可以跳跃的距离是 3，那么表示后面 3 个格子都可以作为 起跳点

2.可以对每一个能作为 起跳点 的格子都尝试跳一次，把 能跳到最远的距离 不断更新

3.如果可以一直跳到最后，就成功了。

代码块

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        max_step = 0
        n =len(nums)
        for i in range(n):
            if i>max_step:
                return False
            max_step = max(max_step,i+nums[i])
            if max_step>=n-1:
                return True
```

## 45.跳跃游戏II

给定一个长度为 `n` 的 **0 索引**整数数组 `nums` 。初始位置在下标 0。

每个元素 `nums[i]` 表示从索引 `i` 向后跳转的最大长度。换句话说，如果你在索引 `i` 处，你可以跳转到任意 `(i + j)` 处：

- `0 <= j <= nums[i]` 且
- `i + j < n`

返回到达 `n - 1` 的最小跳跃次数。测试用例保证可以到达 `n - 1` 。

代码块

```
输入：nums = [2,3,1,1,4]
输出：2
解释：跳到最后一个位置的最小跳跃数是 2。
     从下标为 0 跳到下标为 1 的位置，跳 1 步，然后跳 3 步到达数组的最后一个位置。
```

代码块

```python
class Solution:
    def jump(self, nums: List[int]) -> int:
        #要在一次能跳跃的最大区间内，记录下一次最远可以跳到哪里
        n =len(nums)
        ans =0  #跳跃步数
        max_step =0
        end =0  #表示当前跳跃区间的右端点
        for i in range(n-1):
            max_step = max(max_step,i+nums[i]) #当前跳跃的区间的右端点
            if i==end:
                ans +=1
                end =max_step
        return ans
```

## 1005.K次取反后最大化的数组和

给你一个整数数组 `nums` 和一个整数 `k` ，按以下方法修改该数组：

- 选择某个下标 `i` 并将 `nums[i]` 替换为 `-nums[i]` 。

重复这个过程恰好 `k` 次。可以多次选择同一个下标 `i` 。

以这种方式修改数组后，返回数组 **可能的最大和** 。

**示例 1：**

**输入：** nums = [4,2,3], k = 1
**输出：** 5
**解释：** 选择下标 1 ，nums 变为 [4,-2,3] 。

代码块

```
class Solution:
    def largestSumAfterKNegations(self, nums: List[int], k: int) -> int:
        #先将最小的负数取反
        #如果最后剩余的k为奇数：对nums最小的数取反
        #如果最后剩余的k为偶数：不操作
        nums.sort()
        n =len(nums)
        ans =0
        for i in range(n):
            if nums[i]<0 and k>0:
                nums[i]*=-1
                k-=1    #不要忘记
            if k==0:
                break
        if k%2==1:
            #奇数
            ans = sum(nums)-2*min(nums)
        else:  #偶数/k=0
            ans =sum(nums)
        return ans

```

## 134.加油站

在一条环路上有 `n` 个加油站，其中第 `i` 个加油站有汽油 `gas[i]` 升。

你有一辆油箱容量无限的的汽车，从第 `i` 个加油站开往第 `i+1` 个加油站需要消耗汽油 `cost[i]` 升。你从其中的一个加油站出发，开始时油箱为空。

给定两个整数数组 `gas` 和 `cost` ，如果你可以按顺序绕环路行驶一周，则返回出发时加油站的编号，否则返回 `-1` 。如果存在解，则 **保证** 它是 **唯一** 的。

**示例 1:**

**输入:** gas = [1,2,3,4,5], cost = [3,4,5,1,2]
**输出:** 3
**解释:**
从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有 = 0 + 4 = 4 升汽油
开往 4 号加油站，此时油箱有 4 - 1 + 5 = 8 升汽油
开往 0 号加油站，此时油箱有 8 - 2 + 1 = 7 升汽油
开往 1 号加油站，此时油箱有 7 - 3 + 2 = 6 升汽油
开往 2 号加油站，此时油箱有 6 - 4 + 3 = 5 升汽油
开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。
因此，3 可为起始索引。

这道题的贪心之处在于，假设你从 `start` 出发，走到 `i` 时油不够了，

那我别从 `start` 出发了，从中间某个站 `k` 出发，会不会更好？”

答案是：不会。

为什么？

因为从 `start` 到 `k-1` 这一段，你本来可能还攒了一点油。
你现在把它删掉，从 `k` 重新开始，只会更差，不会更好。

所以：

- 从 `start` 出发都到不了 `i+1`
- 那从 `start+1`、`start+2`、…、`i` 出发，也都到不了 `i+1`

因此这整段都可以直接排除。

所以新的起点只能从：i+1

如果总油量 sum(gas) >= sum(cost)，并且你找到了一个合法起点 start，那么从 start 出发走完 start -> ... -> n-1 这一段后，剩下的油一定足够继续走 0 -> ... -> start-1。

代码块

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        if sum(gas)<sum(cost):
            return -1
        tank =0
        start =0
        n  = len(gas)
        for i in range(n):
            tank +=gas[i]-cost[i]
            if tank<0:
                start =i+1
                tank =0
        return start

```

规则定义： 设学生 A 和学生 B 左右相邻，A 在 B 左边；

**左规则：** 当 ratings B>ratingsA时，B 的糖比 A 的糖数量多。

**右规则：** 当 ratings A>ratings B时，A 的糖比 B 的糖数量多。

相邻的学生中，评分高的学生必须获得更多的糖果 等价于 所有学生满足左规则且满足右规则。

**算法流程：**

先从左至右遍历学生成绩 ratings，按照以下规则给糖，并记录在 left 中：先给所有学生 1 颗糖；

若 ratingsi>ratingsi−1，则第 i 名学生糖比第 i−1 名学生多 1 个。

若 ratings i<=ratings i−1，则第 i 名学生糖数量不变。（交由从右向左遍历时处理。）

经过此规则分配后，可以保证所有学生糖数量 满足左规则 。

同理，在此规则下从右至左遍历学生成绩并记录在 right 中，可以保证所有学生糖数量 满足右规则 。

最终，取以上 2 轮遍历 left 和 right 对应学生糖果数的 最大值 ，这样则 同时满足左规则和右规则 ，即得到每个同学的最少糖果数量。

## 135.分发糖果

`n` 个孩子站成一排。给你一个整数数组 `ratings` 表示每个孩子的评分。

你需要按照以下要求，给这些孩子分发糖果：

- 每个孩子至少分配到 `1` 个糖果。
- 相邻两个孩子中，评分更高的那个会获得更多的糖果。

请你给每个孩子分发糖果，计算并返回需要准备的 **最少糖果数目** 。

**示例 1：**

**输入：** ratings = [1,0,2]
**输出：** 5
**解释：** 你可以分别给第一个、第二个、第三个孩子分发 2、1、2 颗糖果。

代码块

```
class Solution:
    def candy(self, ratings: List[int]) -> int:
        left =[1 for _ in range(len(ratings))]
        right =left[:]
        for i in range(1,len(ratings)):
            if ratings[i] >ratings[i-1]:
                left[i] = left[i-1]+1
        count = left[-1]  #最后一个值要统计下来
        for j in range(len(ratings)-2,-1,-1):
            if ratings[j]>ratings[j+1]:
                right[j] = right[j+1]+1
            count +=max(right[j],left[j])
```

```
        return count
```

## 860.柠檬水找零

在柠檬水摊上，每一杯柠檬水的售价为 `5` 美元。顾客排队购买你的产品，（按账单 `bills` 支付的顺序）一次购买一杯。

每位顾客只买一杯柠檬水，然后**向你付 `5` 美元、`10` 美元或 `20` 美元**。你必须给每个顾客正确找零，也就是说净交易是每位顾客向你支付 `5` 美元。

注意，一开始你手头没有任何零钱。

给你一个整数数组 `bills` ，其中 `bills[i]` 是第 `i` 位顾客付的账。如果你能给每位顾客正确找零，返回 `true` ，否则返回 `false` 。

**示例 1：**

**输入：** bills = [5,5,5,10,20]
**输出：** true
**解释：**
前 3 位顾客那里，我们按顺序收取 3 张 5 美元的钞票。
第 4 位顾客那里，我们收取一张 10 美元的钞票，并返还 5 美元。
第 5 位顾客那里，我们找还一张 10 美元的钞票和一张 5 美元的钞票。
由于所有客户都得到了正确的找零，所以我们输出 true。

代码块

```
class Solution:
    def lemonadeChange(self, bills: List[int]) -> bool:
        #b=20时为什么不收钱：因为 20 元对后续找零没用。这里只记录找零后5，10元还剩多少张，20元的没法给后续找零
        five ,ten =0,0
        for b in bills:
            if b==5:
                five +=1
            elif b==10:
                five -=1
                ten +=1     #不要忘记收钱
            elif ten>0:  #b=20,优先找10元的
                ten -=1
                five -=1
            else:        #b=20，没有10元的，直接找3张5元的
                five -=3
            if five<0:
                return False
        return True
```

## 435.无重叠区间

给定一个区间的集合 `intervals` ，其中 `intervals[i] = [start(i), end(i)]` 。返回 *需要移除区间的最小数量，使剩余区间互不重叠* 。

**注意** 只在一点上接触的区间是 **不重叠的**。例如 `[1, 2]` 和 `[2, 3]` 是不重叠的。

**示例 1:**

**输入:** intervals = [[1,2],[2,3],[3,4],[1,3]]
**输出:** 1
**解释:** 移除 [1,3] 后，剩下的区间没有重叠。

代码块

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        #要移除区间最小数量：转化成，保留最多不重叠的区间
        #要想不重叠的区间多，优先选择结束早的区间保留下来，结束早的区间对后面的区间干扰小
        #贪心的的方法
        ans =0   #要保留的区间数
        pre_r =-inf
        intervals.sort(key = lambda x:x[1])
        for l,r in intervals:
            if l>=pre_r:
                ans +=1
                pre_r = r
        return len(intervals) -ans
```

## 763.划分字母区间

给你一个字符串 `s` 。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。例如，字符串 `"ababcc"` 能够被分为 `["abab", "cc"]` ，但类似 `["aba", "bcc"]` 或 `["ab", "ab", "cc"]` 的划分是非法的。

注意，划分结果需要满足：将所有划分结果按顺序连接，得到的字符串仍然是 `s` 。

返回一个表示每个字符串片段的长度的列表。

代码块

```
输入: s = "ababcbacadefegdehijhklij"
输出: [9,7,8]
解释:
划分结果为 "ababcbaca"、"defegde"、"hijhklij" 。
每个字母最多出现在一个片段中。
像 "ababcbacadefegde", "hijhklij" 这样的划分是错误的，因为划分的片段数较少。
```

代码块

```
class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        ans =[]
        last = {c:i for i,c in enumerate(s)}
        start ,end =0,0
        for i,c in enumerate(s):
            end =max(end,last[c])
            if end ==i:
                ans.append(end-start+1)
                start = i+1
        return ans
```